BIDON · BACKEND · BAC-59

Telemetry POC

Server auction path — events, metrics, traces, and a live funnel

Date 15 September 2026 **Team** Backend **Labels** telemetry, poc **Parent** BAC-59

Status BAC-60 and BAC-61 ready for review; BAC-62–64 in backlog

This proof of concept asks one question: after a single `POST /v2/auction` (and optionally `/v2/show`), can Bidon engineers see which DSPs bid, no-bid, or timed out, whether the ad was shown, and whether the billing notice was accepted — without an SDK change and without standing up the events warehouse.

> **Shape.** Six Linear issues, five merge requests. The parent (BAC-59) is an epic only. Children BAC-60 through BAC-64 each have a Linear story and an MR. Work is stacked: the typed logger first, then auction emit, then traces and the observe stack in parallel, then impression and notice close-out.

1. Why this POC exists

The telemetry PRD and events-warehouse TRD describe the product destination: protobuf on Redpanda, Parquet on Spaces, DuckDB for engineer-mediated SQL. That lake can exist empty. This POC is the **first implementable slice that produces evidence from live auctions**, using Go changes and a local observe stack.

It is server-first on purpose. Certification reconciliation and per-DSP delivery evidence are owned by `bidon-sdkapi`. Waiting on SDK ingest (`POST /v2/telemetry`, remote-config sampling, publisher upgrades) would delay the demo by quarters. At the end of issues 1–5, Bidon can already answer bid / nobid / timeout rate, notice delivery rate, clearing price vs floor, and where auction latency goes.

The existing `ad-events` / `notification-events` streams stay as they are. They carry identifiers and raw DSP payloads that must not become the telemetry product. The POC dual-writes a typed catalog onto a new `telemetry-events` topic.

2. Three grains, one auction

A — Product events

One row per thing that happened. JSON records on Redpanda topic `telemetry-events`. Shared envelope (`event_id`, `event_name`, `auction_id`, …) plus named columns — not a payload blob. Consumed live by RisingWave in this POC; later by the Parquet warehouse.

B — Traces

One tree per auction, child span per DSP. OTLP → otelcol → VictoriaTraces. Answers “show me auction X’s legs.” Sentry stays exceptions-only. `trace_id` is copied onto the event envelope so grains join.

C — Operational metrics

Low-cardinality Prometheus series on `GET /metrics`, scraped into VictoriaMetrics. Answers "is it bad right now." Labels are `dsp` and `outcome` — never `auction_id`. If RisingWave or Victoria* are down, ads still serve and `ad-events` still write.

3. In scope and out of scope

In this POC	Not in this POC
Typed logger on `telemetry-events`	Parquet / S3 / DuckDB / Kafka Connect lake
Bidding-round catalog events + scrapeable counters	Protobuf + Schema Registry
OTel parent + per-DSP child spans	`POST /v2/telemetry` client ingest
Local RisingWave + VM / VT / Grafana	Sampling, Coolify production observe stack
`/v2/show` impression and observed notice HTTP status	Product UI, waterfall fill, bid-match-by-price fix

4. Linear map and merge requests

Six issues, five MRs. File BAC-59 as the epic; BAC-60–64 as children. Each child markdown in `docs/telemetry-poc/` is split into a Linear story (no implementation) and MR instructions for the implementing agent.

```
BAC-59  parent (epic, no MR)
  └── BAC-60  typed telemetry logger          ← MR 1
        └── BAC-61  auction events + metrics  ← MR 2
              ├── BAC-62  traces              ← MR 3  (‖ BAC-63)
              └── BAC-63  RisingWave + VM/VT/Grafana  ← MR 4
                    └── BAC-64  show + notices ← MR 5
```

#	Linear	Title	MR	Blocks on	Status (15 Sep 2026)
0	**BAC-59**	Telemetry POC: server auction path	None (epic)	—	BACKLOG · EPIC
1	**BAC-60**	Typed telemetry-events logger	MR 1 · plumbing only	BAC-59	READY FOR REVIEW
2	**BAC-61**	Auction and DSP events plus /metrics counters	MR 2 · emit + counters	BAC-60	READY FOR REVIEW
3	**BAC-62**	OpenTelemetry auction parent and per-DSP child spans	MR 3 · traces	BAC-60 (pairs with 63)	BACKLOG
4	**BAC-63**	Local RisingWave live funnel plus VM / VT / Grafana	MR 4 · observe stack	BAC-60 (useful after 61)	BACKLOG

#	Linear	Title	MR	Blocks on	Status (15 Sep 2026)
5	**BAC-64**	ad_impression and notice HTTP outcomes	MR 5 · show + notices	BAC-61 and BAC-63	BACKLOG

Linear: BAC-59 · BAC-60 · BAC-61 · BAC-62 · BAC-63 · BAC-64. Stacked git-spice branches match the issue ids (`bac-60-…` , `bac-61-…` , …).

5. Epic — BAC-59 (no MR)

Telemetry POC: server auction path

EPIC

User story. As a Bidon engineer (and later BD / support), I want to observe an auction — which DSPs bid, no-bid, or timed out, whether the ad was shown, and whether the billing notice was accepted.

Goal. One `POST /v2/auction` (and optionally `/v2/show`) proves all three grains: catalog events on `telemetry-events` , low-cardinality series on `/metrics` , a per-auction trace with a child span per DSP.

Definition of done (across children)

- One auction writes `auction_request_received` , per-DSP send/receive, and `auction_completed` to `telemetry-events` .
- Those rows are `SELECT` able in RisingWave (`funnel_5m` , `dsp_outcomes_5m` , `auctions_in_flight`) and on a Grafana board.
- DSP outcome counts and request duration appear on `GET /metrics` and in Grafana → VictoriaMetrics.
- The auction trace (parent + one child per DSP) is in VictoriaTraces; `trace_id` is on the event.
- Optional close-out (BAC-64): same `auction_id` joins to impression and an observed notice HTTP status.
- When telemetry infra is down: `/v2/auction` still succeeds; `ad-events` still written.

No merge request. Compose for the observe services is BAC-63. Coolify/staging observe is called out on the epic as eventual, but the MR for the stack is local-dev first.

6. Issue and MR breakdown

BAC-60 — Typed telemetry-events logger

MR 1 · READY FOR REVIEW

User story. As a backend engineer, I need a logger that writes typed catalog events (shared envelope + event-specific fields) to `telemetry-events`, instead of the unstructured JSON `AdEvent` blobs on `ad-events`.

Goal. `bidon-sdkapi` can fire-and-forget a typed record onto `telemetry-events`. No auction call sites in this issue. `ad-events` stays as it is.

Linear definition of done

- `internal/telemetry` logger: envelope + typed payload, enqueue, do not await Kafka.
- Topic `telemetry-events` wired (`KAFKA_TELEMETRY_EVENTS_TOPIC`).
- Marshalled JSON always has envelope fields (`event_id`, `event_name`, `event_ts`, `schema_version`, `app_id`, `auction_id`, …).
- Missing topic env: log, no panic, caller not blocked.
- `ad-events` producer unchanged.

What the MR delivers

- New package `internal/telemetry`: `Envelope`, `Record`, `Logger` matching the existing event-logger shape (produce in a callback, never wait).
- One JSON object with named columns so RisingWave can `ENCODE JSON`. Unused fields `omitempty` — not a `map[string]any` blob.
- Kafka topic constant + compose / `.env.sample` env. Same broker connection as today's producer; no second client.
- Construct the logger in `cmd/bidon-sdkapi`. Do not instrument `auction.Service` or `bidding.Builder` here.
- No feature flag: if Kafka is on, this logger is on. No protobuf, no Schema Registry, no ingest HTTP API.

Catalog names introduced: `auction_request_received`, `auction_completed`, `dsp_request_sent`, `dsp_response_received`, `dsp_response_rejected`. Schema version `0.1`, sampling rate `1.0`.

Landed on this stack as `aca2ba6` / `1736d66`.

BAC-61 — Auction and DSP events plus /metrics counters

MR 2 · READY FOR REVIEW

User story. As a backend engineer, I want each bidding round to record request, per-DSP send/receive (bid / no-bid / timeout), and completion, plus simple rates I can scrape, so I can see funnel and adapter health without an SDK change.

Goal. Dual-write the server bidding-round catalog to `telemetry_events` and increment low-cardinality Prometheus series. `ad_events` keep working exactly as today.

Linear definition of done

- One 3-DSP auction (bid / nobid / timeout) emits 1 + 3 + 3 + 1 telemetry events.
- DSP latency is send→return, not auction-start→return.
- `GET /metrics` exposes `dsp_response_total{dsp,outcome}`, `dsp_request_duration_seconds{dsp}`, `auction_completed_total{result}`. No `auction_id` label.
- Auction HTTP unchanged if the telemetry logger fails; `ad_events` still written.
- Outcome mapping covered by a table test.

What the MR delivers

- Emit `auction_request_received` at the start of `auction.Service.Run` and `auction_completed` when it finishes (winner, participant count, total latency, short error code).
- In `bidding.Builder.processAdapter`: emit `dsp_request_sent` immediately before the adapter call; set per-demand start/end timestamps; emit `dsp_response_received` (and `dsp_response_rejected` when below floor).
- Thread envelope fields (`app_id`, `session_id`, country, ad type/format) into `BuildParams` — do not geocode again.
- Outcome mapping in `internal/telemetry`: timeout → http_error → malformed → bid → nobid.
- Prometheus counters/histogram on the default registerer already served by Echo.
- Do not change `prepareAuctionRequestEvent` / `prepareBiddingEvents` JSON. Notices, `/v2/show`, OTel spans, waterfall fill are out.

Landed on this stack as `2190636` / `56e238e`. Current branch: `bac-61-poc-auction-and-dsp-events-plus-metrics-counters`.

BAC-62 — OpenTelemetry auction parent and per-DSP child spans

MR 3 · BACKLOG

User story. As a backend engineer (or support looking at one partner complaint), I want the legs of a single auction as one trace so I can see which DSP was slow or timed out without reconstructing it from logs.

Goal. Each `/v2/auction` creates one parent span and one child per DSP. Events from that request carry `trace_id`. When an OTLP endpoint is configured, export there; keep Sentry as it is.

Linear definition of done

- Test recorder: 3 DSPs → 1 parent + 3 children with `app_id`, `auction_id`, `dsp`, `outcome`.
- Telemetry events for that request share the parent `trace_id`.
- `OTEL_EXPORTER_OTLP_ENDPOINT` unset → no OTLP export; auctions unchanged.
- Manual (with BAC-63): tree visible in VictoriaTraces.

What the MR delivers

- When `OTEL_EXPORTER_OTLP_ENDPOINT` is set, add an OTLP HTTP exporter next to the existing Sentry span processor. Do not remove Sentry.
- Parent span `auction.run` in `Service.Run`; child `auction.dsp` around each adapter call. Pass context through `Build` → `HoldAuction` → `processAdapter`.
- Fill `Envelope.TraceID` from the span context at emit sites (BAC-61 may land without this; BAC-62 adds the field fill).
- CI uses `tracetest.SpanRecorder` — otelcol is not required in tests. Destination for a demo is BAC-63.

Accept: auction spans may also show up in Sentry. Unhooking Sentry from this path is a follow-up, not this MR. Out of scope: spanmetrics, sampling, product trace UI.

BAC-63 — Local RisingWave live funnel plus VM / VT / Grafana

MR 4 · BACKLOG

User story. As an engineer demoing telemetry (and later anyone watching auction health), I want to query auctions that just ran — in flight, DSP outcomes, a 5-minute funnel — in a browser or `psql`, without standing up S3 or DuckDB.

Goal. Dev compose runs standalone RisingWave as a consumer of `telemetry-events`, plus VictoriaMetrics, VictoriaTraces, otelcol, and Grafana. sdkapi does not depend on RisingWave. Stopping the observe services does not break ads.

Linear definition of done

- `docker compose -f docker-compose.dev.yml up` starts the new services.
- After one auction (BAC-61): row in Redpanda Console; `SELECT` from `funnel_5m` and `dsp_outcomes_5m`; `auctions_in_flight` populates then clears on `auction_completed`.
- Grafana shows VM rate panels and RisingWave funnel panels.
- With BAC-62: paste `trace_id` into VictoriaTraces and see the tree.
- How-to doc: Console, `psql`, Grafana, VT.
- sdkapi returns 200 for `/v2/auction` if RisingWave / VM / VT are stopped.

What the MR delivers

- Pinned compose services: RisingWave (host 4566), otelcol (4318), VictoriaMetrics (8428), VictoriaTraces (10428), Grafana (3001 — 3010 is bidon-ui).
- Oneshot SQL init: Kafka JSON source matching the BAC-60 column list; materialized views `auctions_in_flight`, `dsp_outcomes_5m`, `funnel_5m`. Joins always on `(app_id, auction_id)`.
- Create topic `telemetry-events` explicitly (auto-create only fires on first produce). Init order: Redpanda healthy → topic → RisingWave → SQL.
- Do **not** add `depends_on: risingwave` to sdkapi. No Connect, Spaces, DuckDB, Alertmanager, or Coolify in this MR.

BAC-64 — ad_impression and notice HTTP outcomes

MR 5 · BACKLOG

User story. As someone reconciling delivery, I want to know whether an auction was shown and whether the DSP accepted the billing/win/loss notice — including HTTP failures — on the same `auction_id` as the bidding-round events.

Goal. `/v2/show` emits `ad_impression` on `telemetry-events`. Notice sends record real HTTP status, retries, and latency, plus a distinct failure event. RisingWave funnel joins through impression and billing. Old `notification-events` stay as they are.

Linear definition of done

- HTTP 500 in a unit test → `notice_delivery_failed` and `success=false`; HTTP 200 → `*_notice_sent` with `http_status=200`.
- Show handler → `ad_impression` produced; BURL still fired.
- `notification-events` JSON shape unchanged.
- After a show on the compose stack: `funnel_5m` impression/billing counts move; `notice_delivery_5m` reflects the HTTP class.

What the MR delivers

- Fix `EventSender`: today any HTTP response is treated as success. Retry transport errors and 5xx (max 3); 4xx is terminal; 2xx is success. Record last status, retry count, and total latency.
- New events: `ad_impression`, `win_notice_sent` (NURL), `loss_notice_sent` (LURL / timeout), `billing_notice_sent` (BURL), `notice_delivery_failed`.
- Thread `app_id` (required on `/v2/show`) so joins work. Keep firing BURL from `HandleShow` — do not move billing onto the telemetry logger.
- Metrics: `notice_delivery_total{type,result}`, `ad_impression_total`.
- Extend RisingWave `funnel_5m` with impressions and successful billing; add `notice_delivery_5m`.

Do not fix float-equality bid match in `notification/handler.go` here — note it as known debt. Out of scope: client-side impression dedupe, `/v2/stats` waterfall events.

7. Catalog and outcomes (issues 1–2)

Event	When	Distinct columns (beyond envelope)
`auction_request_received`	Start of `Run`	`price_floor`
`dsp_request_sent`	Immediately before adapter HTTP	`dsp`, `scope=bidding_round`
`dsp_response_received`	After adapter return / parse	`dsp`, `outcome`, `http_status`, `latency_ms`, `price`
`dsp_response_rejected`	Bid present but below floor	`reject_reason=below_floor`
`auction_completed`	End of `Run`	`winner_dsp`, `price`, `participant_count`, `total_latency_ms`, `error_code`

Event	When	Distinct columns (beyond envelope)
`ad_impression`	`/v2/show` (BAC-64)	`dsp`, `price`
`*_notice_sent` / `notice_delivery_failed`	NURL / LURL / BURL (BAC-64)	`http_status`, `success`, `retry_count`

DSP condition	outcome
`context.DeadlineExceeded`	`timeout`
HTTP 4xx/5xx or status 0 with error	`http_error`
Body / parse failure	`malformed`
`IsBid()`	`bid`
No bid, no error (204 / empty seat)	`nobid`

Precedence: timeout → http_error → malformed → bid → nobid. Envelope joins are always `(app_id, auction_id)` because `auction_id` is client-supplied and not globally unique.

8. How the demo is supposed to look

After BAC-61 (events + counters) and BAC-63 (compose), one seeded auction should prove:

1. **Redpanda Console** `http://localhost:8080` → topic `telemetry-events` has the request, six DSP rows, and completion.
2. **RisingWave** `psql -h localhost -p 4566 -U root -d dev` → `TABLE funnel_5m;` and `dsp_outcomes_5m`; `auctions_in_flight` empties after complete.
3. **Prometheus** `GET /metrics` on sdkapi → `dsp_response_total` and duration histogram moved; Grafana VM panels show rates.
4. **After BAC-62:** copy `trace_id` from an event JSON into VictoriaTraces and see parent + children.
5. **After BAC-64:** `/v2/show` adds impression and billing rows to the same `auction_id`.

Failure mode that must still pass: stop RisingWave / VictoriaMetrics / VictoriaTraces; `curl` auction still 200.

9. Relationship to the rest of telemetry

Document / work	Role versus this POC
PRD BidOn Telemetry v1	Product jobs (fill, render, reconcile, per-DSP loss). POC proves a server-owned subset with zero SDK dependency.
TRD — events warehouse	Parquet on Spaces, 10 min flush, protobuf bus. Downstream of the same catalog; not required to demo the POC.
BAC-53 discovery	Settled the three-grain split and stores. This POC is the first implementation slice of that research.
BID-46 / BID-47 / BID-54	SDK envelope, delivery, and client ingest. Independent once the server logger exists; not on the POC critical path.

Source of truth for filing and implementing: `docs/telemetry-poc/` (README + `00` – `05`). Brief: `docs/telemetry-brief.md`. This PDF is a reading copy of that plan with Linear ids and MR boundaries filled in as of 15 September 2026.